

IoT • AI • SMART HOME

Full Smart Home with AI

An End-to-End IoT Monitoring and AI Analytics Platform

Developed by
AHMED ALMADHOUN

AI Engineering Student
Islamic University of Gaza

ESP32 • Wokwi • MQTT • ThingSpeak • Jupyter Notebook • CSV database

Version 1.0
July 2026



OVERVIEW

Table of Contents

- 01 Project Overview
 - 02 Hardware Architecture & Sensor Network
 - 03 Part One — Real-Time Dashboard Using ThingSpeak
 - 04 Blynk IoT Interface
 - 05 ThingSpeak Dashboard Results
 - 06 Part Two — MQTT to Jupyter Pipeline
 - 07 Jupyter Notebook Implementation
 - 08 Dataset Generation & Storage
 - 09 Software & Hardware Stack
 - 10 Conclusion & Future Work
-

Project summary — The project consists of two independent yet complementary IoT pipelines: The first pipeline streams real-time sensor data from the ESP32 directly to ThingSpeak for cloud-based visualization. The second pipeline transfers sensor readings through the MQTT protocol into a Jupyter Notebook environment, where the data is stored locally, processed, and prepared for AI-based analysis and future prediction models.

01 · OVERVIEW

Project Overview

AI Full Smart Home is a simulated IoT project built on the Wokwi platform around an ESP32 microcontroller. The system reads data from six sensors — temperature, humidity, gas, motion, light, and distance — and transmits the collected sensor data through two independent communication pipelines, independent channels to demonstrate two common IoT communication architectures used in real-world smart systems.

Figure 1 illustrates the complete architecture of the proposed smart home system. The upper section presents the cloud-based visualization pipeline using Blynk IoT and ThingSpeak, while the lower section presents the MQTT-based data acquisition pipeline that feeds a Jupyter Notebook for local storage and AI processing.

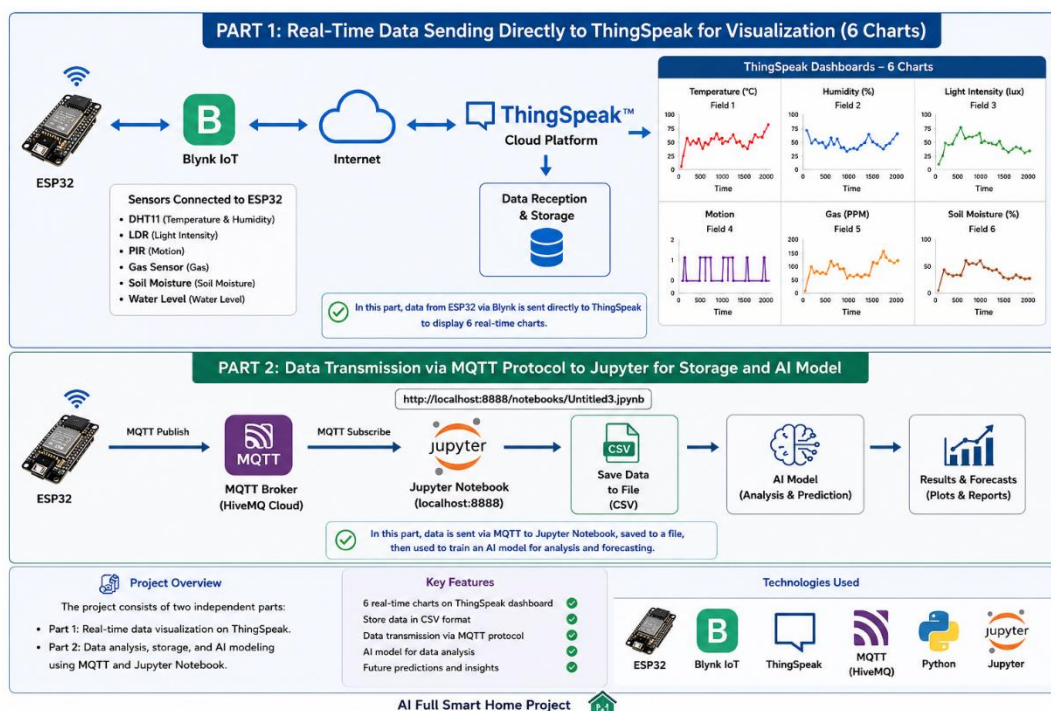


Figure 1 — Complete system architecture: Part One (ThingSpeak) and Part Two (MQTT + AI)

Data Flow — Part One: ESP32 Sensors → Blynk IoT → ThingSpeak Cloud → Live Charts

Data Flow — Part Two: ESP32 Sensors → MQTT Broker → Jupyter Notebook → CSV Storage → AI Model

• Why two separate pipelines?

- Pipeline One** demonstrates reliable real-time monitoring by transmitting sensor data directly to the cloud for instant visualization through ThingSpeak.
- Pipeline Two** focuses on data acquisition by collecting sensor readings through the MQTT protocol, storing them locally as structured datasets, and preparing them for AI and machine learning applications.
- Separating the two pipelines improves modularity, simplifies debugging, and allows each subsystem to evolve independently without affecting the other.

02 · HARDWARE

Hardware Architecture & Sensor Network

The entire circuit is built and simulated inside **Wokwi**, a browser-based simulator for ESP32 / Arduino projects. It allows the full sensor network, wiring, and firmware logic to be tested without any physical hardware, while still connecting to real internet services such as ThingSpeak and MQTT brokers.

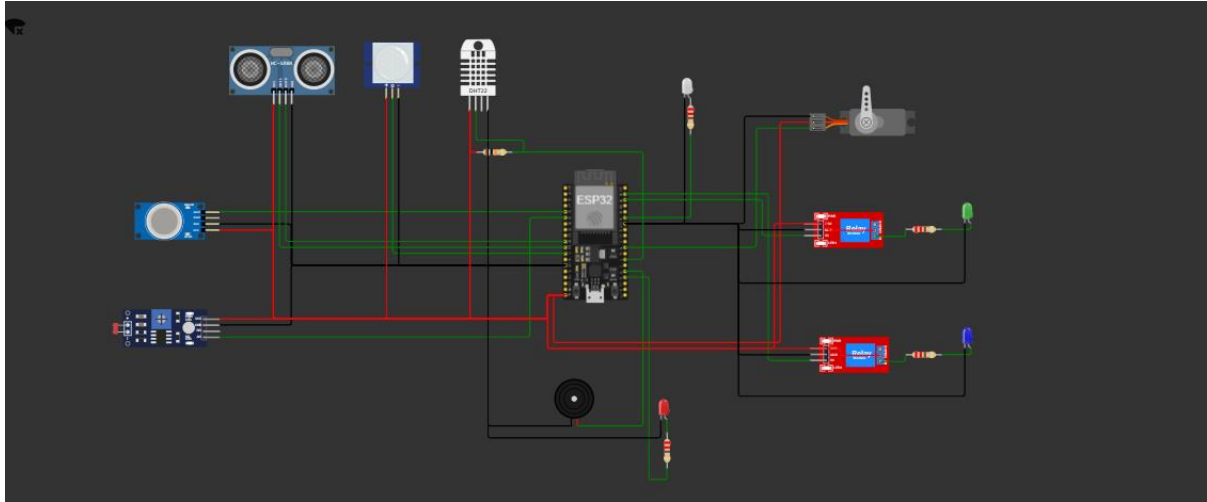


Figure 2 — Wokwi circuit diagram: ESP32 wired to all sensors and actuators

Component	Role in the system
ESP32 Dev Module	Central microcontroller — Wi-Fi, sensor reading, MQTT/HTTP publishing
DHT22 Sensor	Measures ambient temperature and humidity
Ultrasonic Sensor (HC-SR04)	Measures distance for proximity / level detection
MQ-2 Gas Sensor	Detects gas concentration for safety monitoring
PIR Motion Sensor	Detects movement within the monitored area
LDR (Light Sensor)	Measures ambient light intensity
Dual Relay Module	Switch the water pump and fan actuators on/off
Servo Motor	Controls the window
Buzzer + Alarm LED	Local alert for gas, fire, or intrusion conditions

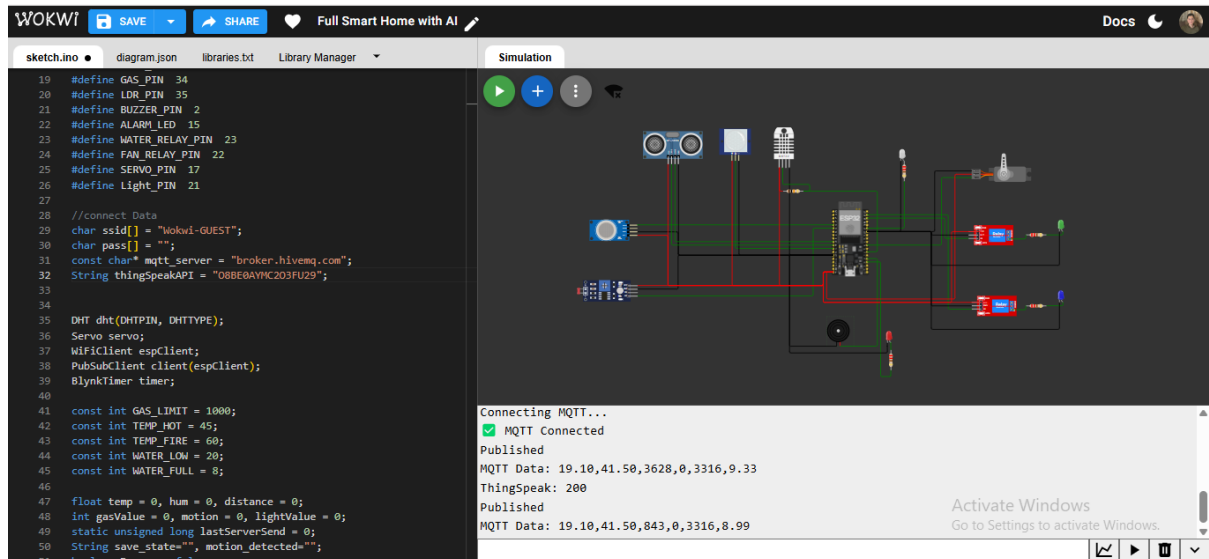


Figure 3 — Wokwi simulation running: firmware connecting to MQTT and publishing

Note — Safety Logic — Critical operating thresholds (gas concentration, temperature, and water level) are implemented directly in the ESP32 firmware. This enables immediate local responses through the buzzer, alarm LED, servo motor, and relay-controlled actuators before cloud services or AI analysis are involved.

01

Part One — Real-Time Dashboard Using ThingSpeak

ESP32 → Blynk IoT → ThingSpeak Cloud

In this first, independent part of the project, the ESP32 firmware connects to Wi-Fi, reads all six sensors, and pushes the readings directly to **ThingSpeak**, a cloud platform for IoT data logging and visualization, using the **Blynk IoT** library for connectivity management. No local processing, database, or AI model is involved here — the goal is purely live, human-readable monitoring.

- **How the data flows**

- The ESP32 samples all sensors on a fixed timer interval.
- Readings are packaged and sent over Wi-Fi through the Blynk IoT connection.
- ThingSpeak receives the data on its channel API and stores it under six separate fields.
- ThingSpeak automatically renders each field as a live, auto-updating line chart.

Each of the six ThingSpeak fields corresponds to one physical sensor: **Temperature**, **Humidity**, **Light**, **Gas**, **Motion**, and **Distance**. Because ThingSpeak channels are public/shareable, this dashboard can be monitored from any browser or device without installing anything.

Result — The ThingSpeak channel receives sensor readings approximately every few seconds—in parallel with the Blynk IoT platform—demonstrating a stable end-to-end connection between them and the ESP32.

Blynk IoT Interface

The ESP32 controller connects to the Blynk IoT platform, which provides a real-time interface for monitoring sensor values and manually controlling actuators; the interface displays live sensor readings while allowing the user to operate the window servo motor, fan, lighting, and water pump, as well as select system modes directly via the app.

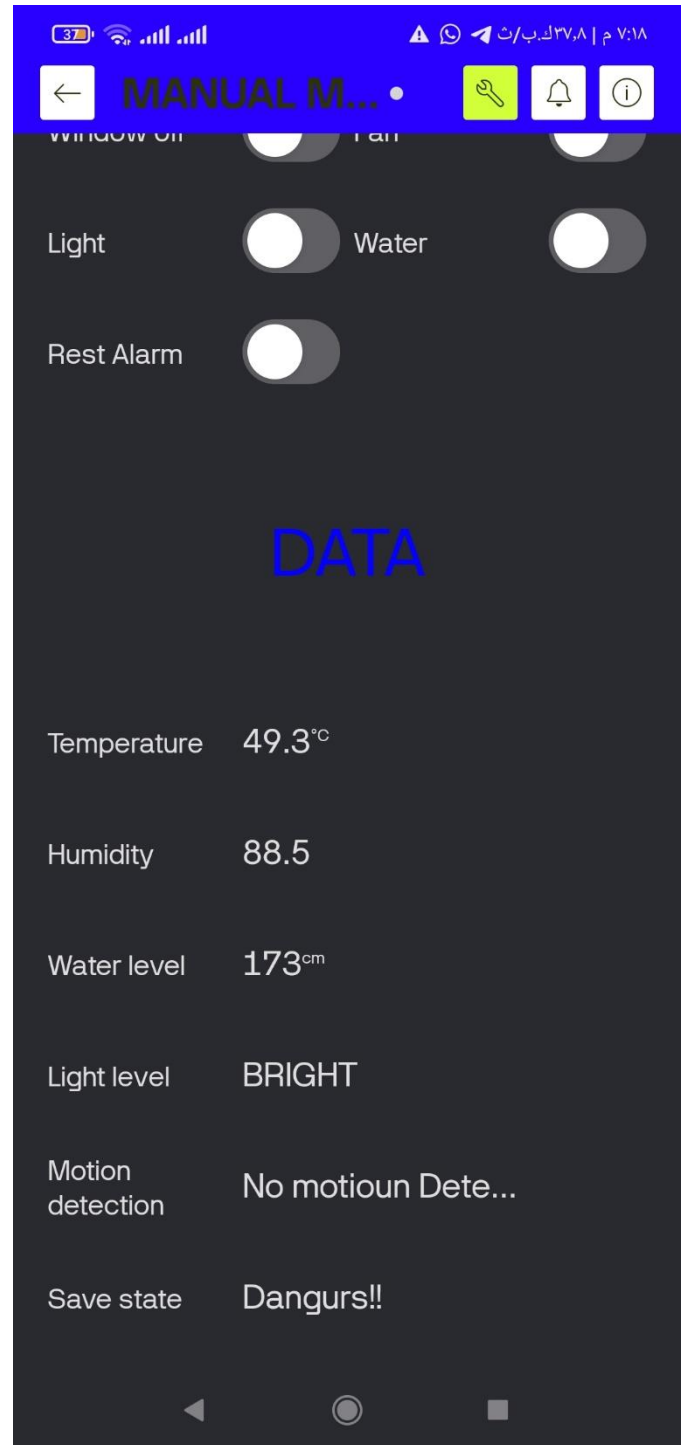
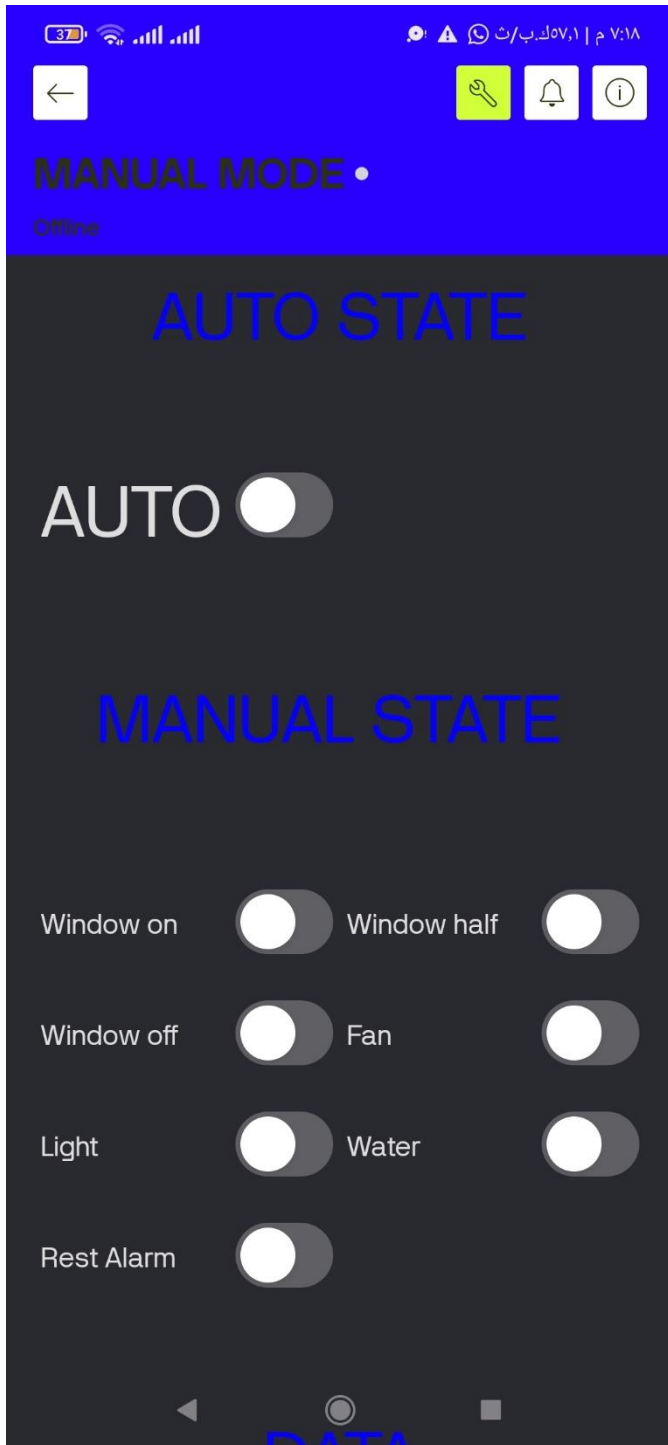


Figure 4 and Figure 5 — The two images show the interface of the Blynk IoT application, which includes control buttons and sensor information.

ThingSpeak Channel Results

The screenshots below were captured directly from the live "Full Smart Home with AI" channel on ThingSpeak, showing all six fields updating in real time as the Wokwi simulation ran.

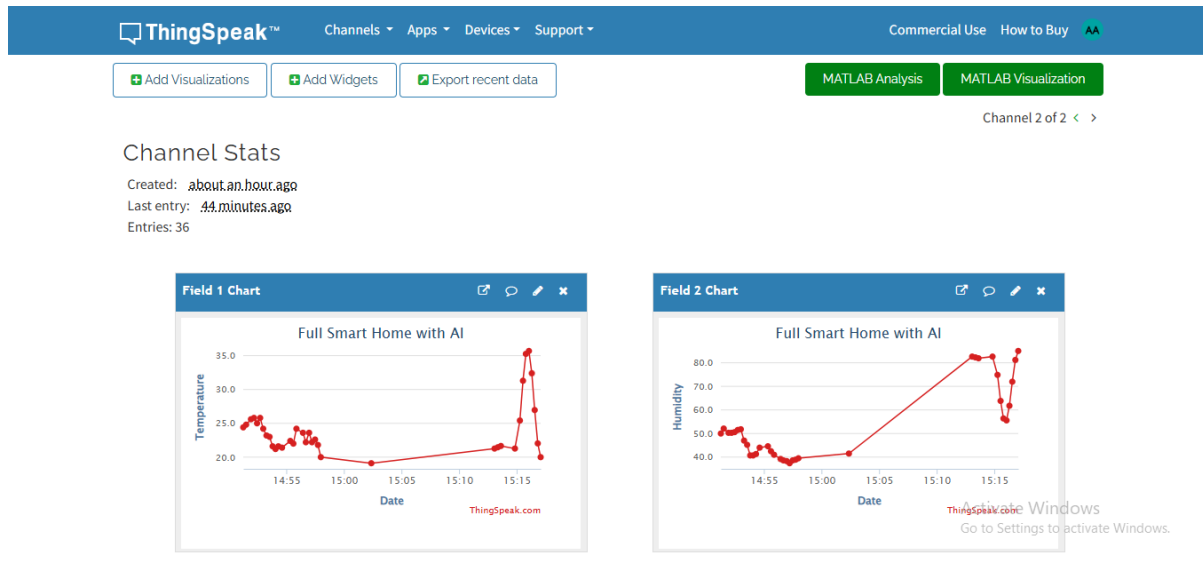


Figure 6 — Field 1: Temperature (°C) and Field 2: Humidity (%)

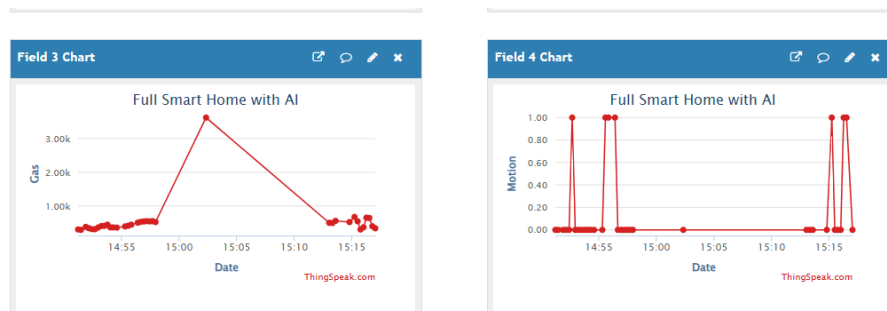


Figure 7 — Field 3: Gas concentration and Field 4: Motion detection (0/1)

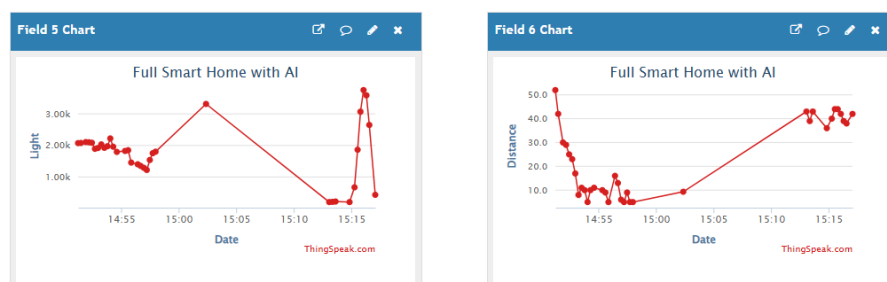


Figure 8 — Field 5: Light intensity and Field 6: Distance (cm)

- **Reading the charts**
- **The motion** detection graph switches cleanly between binary states (0 and 1), confirming reliable PIR sensor operation.
- **All six sensor channels** are synchronized on the same transmission cycle, demonstrating stable real-time cloud communication.

02

Part Two — MQTT to Jupyter Notebook Pipeline

ESP32 → MQTT Broker → Jupyter Notebook → CSV → AI Model

The second—completely independent—track focuses on collecting reliable data for artificial intelligence applications; an ESP32 controller transmits sensor data via the MQTT protocol to a Python environment running within a Jupyter Notebook, where the data is processed, stored, and prepared for machine learning tasks.

- **Pipeline stages**

- **Publish** — The ESP32 unit sends sensor readings to a specific topic on a public MQTT broker.
- **Subscribe** — a Python script running inside Jupyter Notebook subscribes to the same topic using the `paho-mqtt` library.
- **Parse** — each incoming MQTT message is decoded and split into its six numeric values.
- **Store** — the parsed values are appended as a new row to a local `sensor_data.csv` file using `pandas`.
- **Analyze** — the accumulated CSV dataset becomes the input for an AI/ML model that can detect patterns and generate predictions.

Running this pipeline required launching a local Jupyter server and installing the necessary Python packages (``paho-mqtt``, ``pandas``, and ``matplotlib``).

Jupyter Notebook Implementation

A local Jupyter server was launched to host the notebooks used for subscribing to the MQTT protocol and processing data.

```

npm prefix
2026-07-04 15:05:43.007 ServerApp Extensions will be fetched using proxy, proxy host and port: ('10.10.00.00', '80')
2026-07-04 15:05:43.009 ServerApp jupyterlab | extension was successfully loaded.
2026-07-04 15:05:43.065 ServerApp notebook | extension was successfully loaded.
2026-07-04 15:05:43.066 ServerApp Serving notebooks from local directory: C:\Users\Eiz
2026-07-04 15:05:43.066 ServerApp Jupyter Server 2.0.0 is running at:
2026-07-04 15:05:43.066 ServerApp http://localhost:8888/tree?token=ef1dca9602996e254d1c2f188480873f377bd085264437f4
2026-07-04 15:05:43.066 ServerApp http://127.0.0.1:8888/tree?token=ef1dca9602996e254d1c2f188480873f377bd085264437f4
2026-07-04 15:05:43.066 ServerApp Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).

To access the server, open this file in a browser:
file:///C:/Users/Eiz/AppData/Roaming/jupyter/runtime/jpserver-2196-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=ef1dca9602996e254d1c2f188480873f377bd085264437f4
http://127.0.0.1:8888/tree?token=ef1dca9602996e254d1c2f188480873f377bd085264437f4
2026-07-04 15:05:43.067 ServerApp Skipped non-installed server(s): basedpyright, bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-lang
server, jedi-language-server, julia-language-server, pyrefly, pyright, python-language-server, python-lsp-server, r-language-server, sql-language-server, texlab, typescr
ipt-language-server, unified-language-server, vscode-css-language-server-bin, vscode-html-language-server-bin, vscode-json-language-server-bin, yamll-language-server
2026-07-04 15:05:43.067 ServerApp Kernel started: 73b9e875-8270-4f3b-9e49-f02ac8bbb42a
[IPKernelApp] WARNING | Kernel is running over TCP without encryption. All communication (including code and outputs) is sent in plain text and is susceptible to eavesd
ropping. Use IPC transport or launch with kernel manager-provisioned CurveZMQ keys to enable transport encryption.
2026-07-04 15:06:34.963 ServerApp Adapting from protocol version 5.3 (kernel 73b9e875-8270-4f3b-9e49-f02ac8bbb42a) to 5.4 (client).
2026-07-04 15:06:34.964 ServerApp Connecting to kernel 73b9e875-8270-4f3b-9e49-f02ac8bbb42a.
2026-07-04 15:06:34.981 ServerApp The websocket_ping_timeout (90000) cannot be longer than the websocket_ping_interval (30000).
Setting websocket_ping_timeout=30000
2026-07-04 15:06:34.987 ServerApp Adapting from protocol version 5.3 (kernel 73b9e875-8270-4f3b-9e49-f02ac8bbb42a) to 5.4 (client).
2026-07-04 15:06:34.988 ServerApp Connecting to kernel 73b9e875-8270-4f3b-9e49-f02ac8bbb42a.
2026-07-04 15:06:34.993 ServerApp Adapting from protocol version 5.3 (kernel 73b9e875-8270-4f3b-9e49-f02ac8bbb42a) to 5.4 (client).
2026-07-04 15:06:34.994 ServerApp Connecting to kernel 73b9e875-8270-4f3b-9e49-f02ac8bbb42a.

```

Figure 9 — Jupyter server startup: local access URL and token

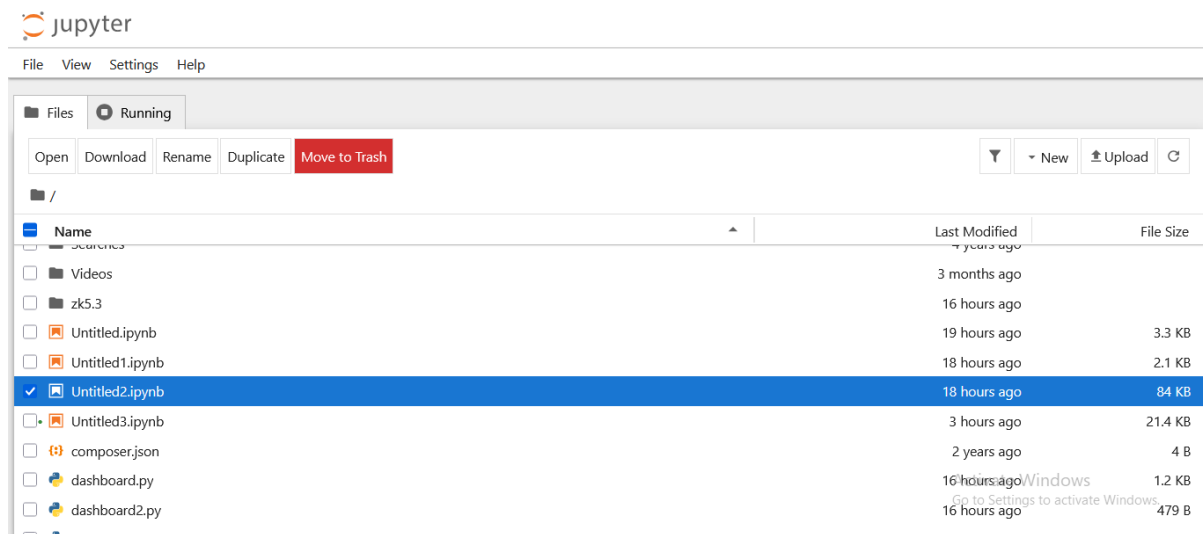


Figure 10 — Jupyter file browser showing the project notebooks

The screenshot shows the JupyterLab interface with a notebook titled 'Untitled3'. The last checkpoint was 19 hours ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The notebook contains three code cells:

```
[14]: import sys
      print(sys.executable)

C:\Users\Eiz\AppData\Local\Programs\Python\Python314\python.exe

[2]: import os
     print(os.getcwd())

C:\Users\Eiz

[24]: import sys
      !{sys.executable} -m pip install matplotlib
```

The output of the third cell shows the following requirements being satisfied:

```
Requirement already satisfied: matplotlib in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.5.0)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (12.3.0)
```

Figure 11 — Verifying the Python environment and installing matplotlib

The screenshot shows the JupyterLab interface with a notebook titled 'Untitled3'. The last checkpoint was 19 hours ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The notebook contains two code cells:

```
[16]: import sys
      !{sys.executable} -m pip install paho-mqtt pandas

Requirement already satisfied: paho-mqtt in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.1.0)
Requirement already satisfied: pandas in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.0.3)
Requirement already satisfied: numpy>=2.3.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[18]: import sys
      !{sys.executable} -m pip install paho-mqtt pandas

Requirement already satisfied: paho-mqtt in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.1.0)
Requirement already satisfied: pandas in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.0.3)
Requirement already satisfied: numpy>=2.3.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

Figure 12 — Installing paho-mqtt and pandas for MQTT and data handling

With the environment ready, the notebook defines an `on_message` callback that fires every time a new MQTT message arrives, decodes the payload, splits it into six sensor values, and appends the row to the CSV dataset using `pandas.DataFrame.to_csv` in append mode.

```
[ ]: import paho.mqtt.client as mqtt
import pandas as pd
import os
from datetime import datetime

broker = "broker.hivemq.com"
topic = "Registernumber"

file_name = "sensor_data.csv"

# إذا الملف غير موجود، أنشئه مع أسماء الأعمدة
if not os.path.exists(file_name):
    df = pd.DataFrame(columns=[
        "Time",
        "Temperature",
        "Humidity",
        "Gas",
        "Motion",
        "Light",
        "Distance"
    ])
    df.to_csv(file_name, index=False)

def on_message(client, userdata, msg):
    data = msg.payload.decode()
    print("📡", data)

    values = data.split(",")

    if len(values) != 6:
        return
```

Activate Windows
Go to Settings to activate Windows.

```
    if len(values) != 6:
        return

    row = {
        "Time": datetime.now().strftime("%H:%M:%S"),
        "Temperature": float(values[0]),
        "Humidity": float(values[1]),
        "Gas": int(values[2]),
        "Motion": int(values[3]),
        "Light": int(values[4]),
        "Distance": float(values[5])
    }

    df = pd.DataFrame([row])
    df.to_csv(file_name, mode="a", header=False, index=False)

client = mqtt.Client()
client.on_message = on_message

client.connect(broker, 1883, 60)
client.subscribe(topic)

print("📡 Saving data to sensor_data.csv ...")

client.loop_forever()
```

```
C:\Users\Eiz\AppData\Local\Temp\ipykernel_12888\2428788591.py:46: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client()

📡 Saving data to sensor_data.csv ...
📡 31.90,55.50,843,1,3092,11.68
📡 10 00 55 50 843 1 3092 11 68
```

Activate Windows
Go to Settings to activate Windows.

Figure 13 and Figure 14 — The two previous images contain Python code that receives data from ESP32 sensors via the MQTT protocol and stores it in a CSV dataset.

Dataset Generation & Storage

When the Python code is executed and the `client.loop_forever()` function is called, the notebook begins continuously listening to the MQTT topic, printing all incoming sensor readings while simultaneously saving them in a CSV dataset.

```
client.connect(broker, 1883, 60)
client.subscribe(topic)

print(" Saving data to sensor_data.csv ...")

client.loop_forever()
```

C:\Users\Eiz\AppData\Local\Temp\ipykernel_12888\2428788591.py:46: DeprecationWarning: Callback API version 1 is deprecated, update to latest version

```
client = mqtt.Client()
Saving data to sensor_data.csv ...
31.90,55.50,843,1,3092,11.68
19.00,55.50,843,0,2800,11.56
19.00,55.50,843,1,2800,11.97
19.00,55.50,843,1,2800,11.56
19.00,55.50,843,1,2800,11.56
3.60,55.50,843,1,3413,24.00
3.60,55.50,843,1,3413,23.97
3.60,55.50,843,1,3413,24.00
```

Activate Windows

Figure 15 — Live MQTT stream

After operating for a period of time, the CSV file accumulates a series of data from the six sensors. The data in this file was loaded using the `pandas.read_csv` function to verify its structure and serve as the foundation for the AI-based analysis phase.

```
[2]: import pandas as pd

df = pd.read_csv("sensor_data.csv")

print(df.tail(20))
print("عدد الصفوف:", len(df))
```

	Time	Temperature	Humidity	Gas	Motion	Light	Distance
130	21:56:43	20.1	47.5	868	0	4023	1.97
131	21:56:48	20.1	47.5	868	0	4023	1.97
132	21:56:54	20.1	47.5	868	0	4023	1.97
133	21:57:00	20.1	47.5	868	0	4023	1.97
134	21:57:30	20.1	47.5	868	0	4023	1.97
135	21:57:35	20.1	47.5	868	0	4023	1.97
136	21:57:50	20.1	47.5	868	0	4023	1.97
137	21:57:50	20.1	47.5	868	0	4023	1.97
138	21:57:53	20.1	47.5	868	0	4023	1.97
139	21:57:58	20.1	47.5	868	0	4023	1.97
140	21:58:04	20.1	47.5	868	0	4023	1.97
141	21:58:16	20.1	47.5	868	0	4023	1.97
142	21:58:29	20.1	47.5	868	0	4023	1.97
143	21:58:29	20.1	47.5	868	0	4023	1.97
144	21:58:29	20.1	47.5	868	0	4023	1.97
145	11:20:50	49.3	88.5	843	0	1001	114.02

Activate Windows
Go to Settings to activate Windows.

Figure 16 — Loading and inspecting the saved dataset: Time, Temperature, Humidity, Gas, Motion, Light, Distance

• Dataset columns

- **Time** — time of the reading
- **Temperature** — degrees Celsius, from the DHT22 sensor
- **Humidity** — relative humidity percentage, from the DHT22 sensor
- **Gas** — raw analog reading from the MQ-2 gas sensor
- **Motion** — binary flag (0/1) from the PIR sensor
- **Light** — raw analog reading from the LDR sensor
- **Distance** — centimeters, from the ultrasonic sensor

Next step — The previously stored dataset constitutes the exact format required to train an AI model; for instance, a model designed to detect potential risks before they occur by making predictions based on patterns learned from the archived data on such risk scenarios.

Software & Hardware Stack

The project combines simulated hardware, cloud IoT services, and a Python-based analytics environment. The table below summarizes every technology used across both parts of the system.

ESP32 Main microcontroller	Wokwi Browser-based circuit simulator for the full sensor network	Blynk IoT Connectivity layer between ESP32 and the cloud in Part One
ThingSpeak Cloud channel storing and charting the 6 live sensor fields	MQTT Publish/subscribe protocol carrying data in Part Two	Python Core language for the data pipeline and analysis notebook
Jupyter Notebook Interactive environment hosting the MQTT client and analysis code	pandas Structuring, appending, and reading the sensor CSV dataset	paho-mqtt Python MQTT client library used to subscribe to the broker

Layer	Part One	Part Two
Transport	HTTP and Wifi	MQTT publish/subscribe
Destination	ThingSpeak cloud channel	Local Jupyter Notebook
Storage	ThingSpeak time-series database	Local sensor_data.csv
Output	6 live auto-updating charts	Structured dataset for AI modeling
Best for	Instant, shareable monitoring	Analysis, training, and prediction

Conclusion & Future Work

This project successfully demonstrates two complementary approaches to processing IoT sensor data: a real-time display via ThingSpeak and a control panel via Blynk IoT, and a structured, storable data channel via MQTT and Jupyter Notebook. Together, these approaches cover a wide range of applications, from **real-time monitoring** to **data-driven intelligence**.

- **Key achievements**

- Simulated a complete six-sensor smart-home circuit on Wokwi, including safety actuators (relays, buzzer, alarm LED, servo).
- Established a stable, continuous connection from the ESP32 to a public ThingSpeak channel with six live charts.
- Built an independent MQTT pipeline delivering the same sensor data into a Python environment.
- Implemented an automated CSV logging system inside Jupyter Notebook using pandas.
- Prepared a clean, time-stamped dataset ready for AI model training.

- **Future work**

- Train an anomaly-detection model on the gas and motion fields to trigger automatic alerts.
- Add a predictive model that pre-emptively controls the fan and water relays based on temperature and humidity trends.
- Replace the temporary MQTT public broker with a persistent, secured broker for long-term deployment.
- Deploy the trained AI model back onto the ESP32 or an edge device for fully autonomous operation.

"AI Full Smart Home" bridges simulation and real-world IoT engineering — proving that a single sensor network can serve both a human-facing live dashboard and a machine-facing AI training pipeline, without either one interfering with the other.